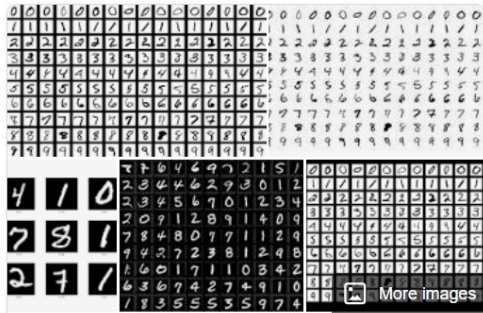




MNIST database

The MNIST database is a large database of handwritten digits that is commonly used for training various image processing systems. The database is also widely used for training and testing in the field of machine learning. It was created by "re-mixing" the samples from NIST's original datasets. [Wikipedia](#)

✦ MNIST = Modified National Institute of Standards and Technology database

Importieren der Python-Pakete

den mnist-Datensatz aus keras.datasets holen

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# get the mnist dataset for keras.datasets
mnist = tf.keras.datasets.mnist
```

✦ Aufteilung des Datensatzes in Training und Test

Erstellen eines Modells für ein neuronales Netz

```
#Splitting of the dataset into training and testing

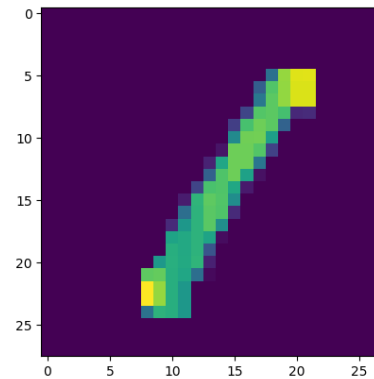
(x_train, y_train), (x_test, y_test) = mnist.load_data()

plt.imshow(x_train[0])
#print(x_train[0])

x_train = tf.keras.utils.normalize(x_train, axis=1)
x_test = tf.keras.utils.normalize(x_test, axis=1)

plt.imshow(x_train[3]) #, cmap=plt.cm.binary
plt.show()
#print(x_train[0])
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 1s 0us/step



Beginnen Sie mit dem Programmieren oder [generieren](#) Sie Code mit KI.

✦ Erstellen eines Modells für ein neuronales Netz

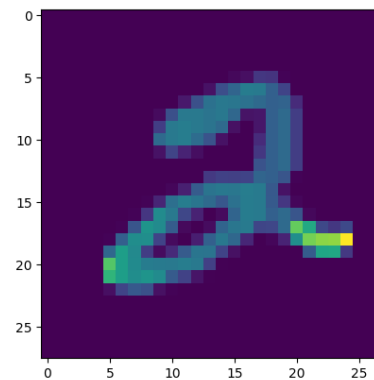
Creating a model for forward feed network

```
model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(64, activation='relu'))
model.add(tf.keras.layers.Dense(128, activation='relu'))
model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/
super().__init__(**kwargs)

✦ Plot the input digit

```
#plt.imshow(x_train[2], cmap=plt.cm.binary)
plt.imshow(x_train[5])
plt.show()
#print(x_train[2])
```



✦ Die Modellkompilierung durch Optimierung.

Optimierungsparameter: Verlust und Genauigkeit

```
# The model compilation through optimize. The loss and accuracy are used
# as a parameters for optimization process.
```

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

das Modell mit dem Trainingsdatensatz trainieren

```
# We train the model with training dataset. The number of iteration is 3.
```

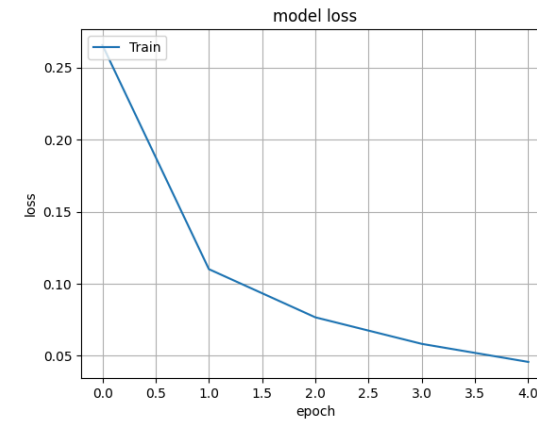
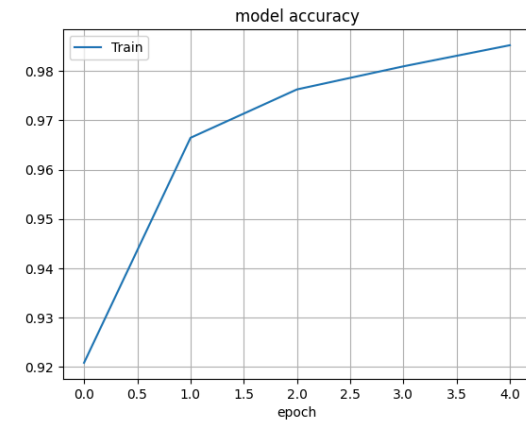
```
history=model.fit(x_train,y_train, epochs=5)
```

```
Epoch 1/5
1875/1875 ————— 10s 5ms/step - accuracy: 0.9208 - loss: 0.2655
Epoch 2/5
1875/1875 ————— 9s 5ms/step - accuracy: 0.9665 - loss: 0.1101
Epoch 3/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.9763 - loss: 0.0767
Epoch 4/5
1875/1875 ————— 9s 5ms/step - accuracy: 0.9810 - loss: 0.0583
Epoch 5/5
1875/1875 ————— 9s 5ms/step - accuracy: 0.9852 - loss: 0.0458
```

Plotting the accuracy and Loss curve

```
plt.plot(history.history['accuracy'])
#plt.plot(history.history['loss'])
#plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.grid()
plt.gcf()
plt.legend(['Train'], loc='upper left')
plt.grid()
plt.show()

# summarize history for loss
plt.plot(history.history['loss'])
#plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.grid()
plt.show()
```



Validierung des Modells

```
val_loss, val_accuracy = model.evaluate(x_test,y_test)
print(val_loss,val_accuracy)
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.9718 - loss: 0.0973
0.09728512167930603 0.9718000292778015
```

Das Modell speichern

```
# model.save('mnist_num_model')
```

das Modell zu laden

```
#new_model = tf.keras.models.load_model('mnist_num_model')
```

Testdatenvorhersage

```
#prediction = new_model.predict([x_test])
```

```
prediction = (model.predict([x_test]))
```

```
88/313 ————— 0s 2ms/step/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: Expected: keras_tensor
Received: inputs=('Tensor(shape=(32, 28, 28))',)
warnings.warn(msg)
313/313 ————— 1s 2ms/step
/usr/local/lib/python3.12/dist-packages/keras/src/models/functional.py:241: UserWarning: The structure of `inputs` doesn't m
Expected: keras_tensor
Received: inputs=('Tensor(shape=(None, 28, 28))',)
warnings.warn(msg)
```

✓ Vergleichen Sie die Testeingabe und -ausgabe

```
print(np.argmax(prediction[3]))
plt.imshow(x_test[3])
plt.show()
```

